

概述

本手册适用范围：

适用范围
SPC1169, SPD1179, SPD1176

当前 E²PROM 特指以较小单元(比如 word)作为基础操作单位的存储器,容量通常不超过 512K。相比 Flash,其不需要为了修改某个较小单元而将整个扇区读取,擦除,修改,重新写入,从而具有较小单元数据修改迅速,整体使用寿命长的两大特点。

- 文中示例代码中有关 Flash 及 RAM 地址范围的描述需要根据不同的产品进行更改;
- 采用多个 Page 软件模拟 E²PROM。

目录

1	外部 E²PROM 和模拟 E²PROM 之间的主要区别	7
1.1	E ² PROM 时序	7
2	实现 E²PROM 模拟	8
2.1	算法简介	8
2.1.1	EEPROM 实现	8
2.1.2	MEEPROM 实现	9
2.2	Page 格式	10
2.3	磨损均衡算法	11
2.4	原理示例说明	12
2.5	E ² PROM 固件说明	13
2.6	E ² PROM 模拟算法线程安全	14
3	异常概述以及对应解决方案	15
3.1	EEPROM_STATUS_PARITY_ERROR	16
3.2	EEPROM_STATUS_INVALID_ADDR	16
3.3	EEPROM_STATUS_NO_DATA	17
3.4	EEPROM_STATUS_INVALID_CB	17
3.5	EEPROM_STATUS_INVALID_HEADER	17
3.6	EEPROM_STATUS_NO_PAGE_FOUND	18
3.7	EEPROM_STATUS_ADDR_MISMATCH	18
3.8	EEPROM_STATUS_TRANSFER_ERROR	18
3.9	EEPROM_STATUS_WRITE_ERROR	19
3.10	EEPROM_STATUS_ERASE_ERROR	19
3.11	EEPROM_STATUS_PAGE_HEADER_ERROR	19
3.12	EEPROM_STATUS_ELEMENT_NOT_EMPTY	19
3.13	EEPROM_STATUS_WRITE_CHECK_FAIL	20
3.14	EEPROM_STATUS_INVALID_ENTRY	20
4	固件对应解决方案	21
5	工程应用讨论	23
5.1	掉电保存数据	23
5.1.1	主动掉电	23
5.1.2	随机掉电	24
5.2	数据粒度管理	25
5.3	XIP 关闭期间中断处理	25

5.4	看门狗不断复位或芯片调试接口被锁定	25
5.5	中断时间开销	26
5.5.1	同一中断不断触发以及对应解决方案	26
5.5.2	不同中断先后触发以及对应解决方案	27

SPIN TROL

图片列表

图 2-1: EEPROM 的 Page0, Page1 与 Page2	8
图 2-2: MEEPROM 的 Page0 与 Page1.....	9
图 2-3: E ² PROM 模拟算法中的变量格式.....	10
图 2-4: 数据更新过程	12
图 4-1: 函数 ReadWord.....	21
图 4-2: 函数 WriteWord.....	21
图 4-3: 函数 Init.....	22
图 4-4: 函数 Format	22
图 5-1: 中断 A 先后触发.....	26
图 5-2: 中断 A, B 先后触发.....	27
图 5-3: 在中断 A 中清中断 B Flag	27

表格列表

表 1-1: 外部 E ² PROM 和模拟 E ² PROM 之间的区别	7
表 1-2: 模拟 E ² PROM 时序 (SYSCLK = 100MHz)	7
表 2-1: E ² PROM 模拟算法	8
表 2-2: 变量虚拟地址	12
表 2-3: E ² PROM 模拟算法 API 说明	13
表 2-4: 冲突表函数	14
表 2-5: 冲突避免方法	14
表 3-1: E ² PROM 模拟算法 API 返回值说明	15
表 5-1: V _{DVDD33} 推荐工作条件	24

版本历史

版本	日期	作者	状态	变更
A/0	2023-03-10	黄恒方	Outdated	1. 首次发布。
A/1	2023-12-06	柴灿	Outdated	1. 内容描述改为 MEEPROM。
C/0	2024-02-02	苏杭	Outdated	1. 更新 章节 1 ， 章节 2 ， 章节 0 ， 章节 4 ， 章节 5 。
C/1	2024-05-30	苏杭	Outdated	1. 更新 章节 1 ， 章节 2 ， 章节 0 ， 章节 4 ， 章节 5 。
C/2	2024-06-05	苏杭	Released	1. 更新 章节 5.1.2

1 外部 E²PROM 和模拟 E²PROM 之间的主要区别

为了减少所用元件，节省 PCB 空间和降低系统成本，Flash 存储器可以通过软件模拟成 E²PROM。

模拟 E²PROM 和外部串行 E²PROM 之间的主要区别在于写入时间是否固定以及是否依赖 MCU 供电，如表 1-1 所示。可以看到，模拟 E²PROM 先天具有写操作时间不固定的特点，如果不在应用端加以处理，见章节 5.1.2，其不具有外部 E²PROM 较小单元数据修改迅速的特点。

表 1-1：外部 E²PROM 和模拟 E²PROM 之间的区别

特性	外部 E ² PROM（例如，AT24C02：I2C 串行访问 E ² PROM）	利用片上 Flash 模拟 E ² PROM
写操作时间	写一个 Byte：5ms 以内 因此，写一个 Word = 20ms Page（8Bytes）写：5ms 以内 因此，写一个 Word = 2.5ms	写一个 Word（32-bit）：从 50us 到 18ms
写方式	启动后，不依赖 MCU（VDD33）只需要供电	一旦启动，则依赖 MCU（VDD33）
读访问时间	读一个 Word（32-bit）：66us	读一个 Word（32-bit）：3us（SYSCLK = 100MHz）
写/擦除周期	100 万次	每个 Flash Page 10 万次

1.1 E²PROM 时序

本章节介绍 SPC1169 中模拟 E²PROM 相关的时序参数，如表 1-2 所示。

表 1-2：模拟 E²PROM 时序（SYSCLK = 100MHz）

操作	模拟 E ² PROM 时序		
	最小	典型	最大
WriteWord（典型工况）	-	50us	-
WriteWord ^[1] （伴有 Flash Page 数据搬移工况）	-	18ms	-
ReadWord	-	3us	-

[1] 基于 Page 中 Sector 数目为 1，且虚拟地址范围为 0~255 的场景测得。

2 实现 E²PROM 模拟

2.1 算法简介

对于 SPC1169 而言，共有两种 E² PROM 模拟算法，名称分别为 EEPROM、MEEPROM，如表 2-1 所示。

表 2-1: E² PROM 模拟算法

	EEPROM	MEEPROM
算法所用 Page 位置	RDN 区域	Main Array 或 RDN 区域
算法所用 Page 页数	3 页	2 页
Page 中 Sector 数目	1	可调，通常设为 1
Page 中虚拟地址范围	0 ~ 255	可调，通常设为 0 ~ 255

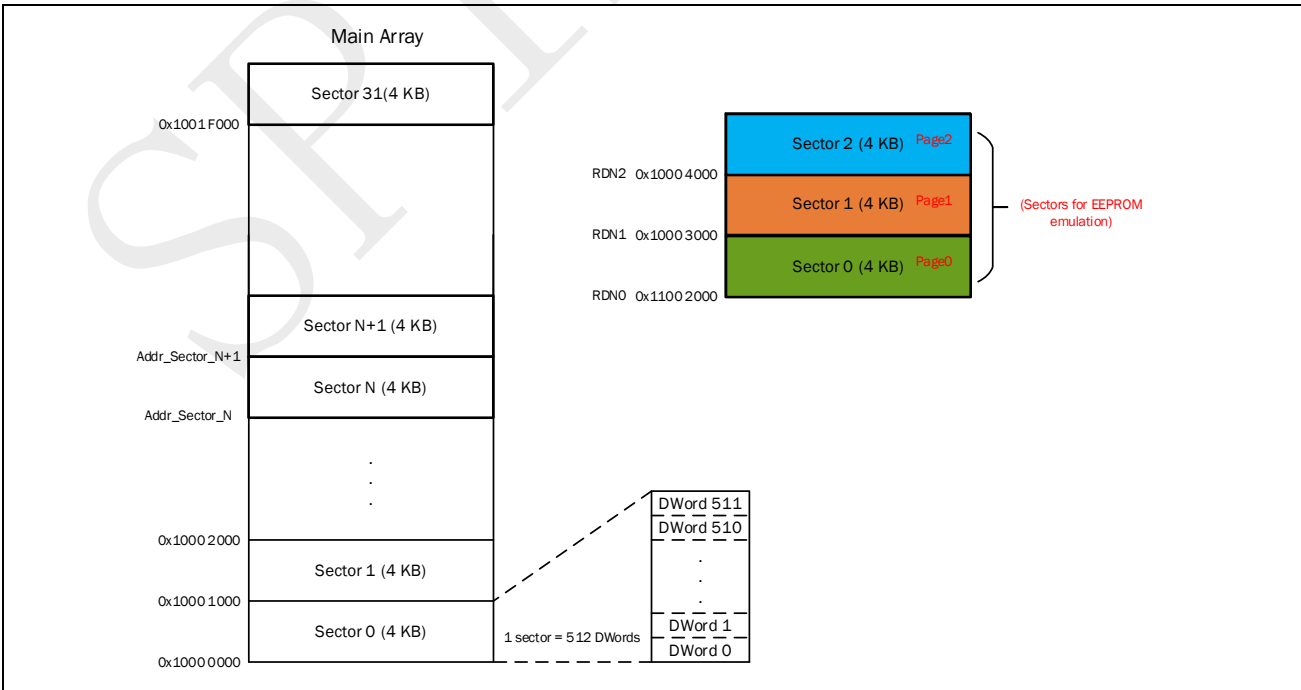
EEPROM 本身不占用 Main Array 空间，且算法模拟的 page 数目更多，整体使用寿命更长，更加推荐使用。

MEEPROM 具有配置更加灵活，占用空间小的特点。

2.1.1 EEPROM 实现

EEPROM 使用 3 个固定 Page（RDN0，RDN1，RDN2）来实现 E² PROM 模拟，每个 Page 由 1 个 Flash Sector 组成，如图 2-1 所示。

图 2-1: EEPROM 的 Page0，Page1 与 Page2



[1] DWord = Double Word, 1 DWord = 64-bit。

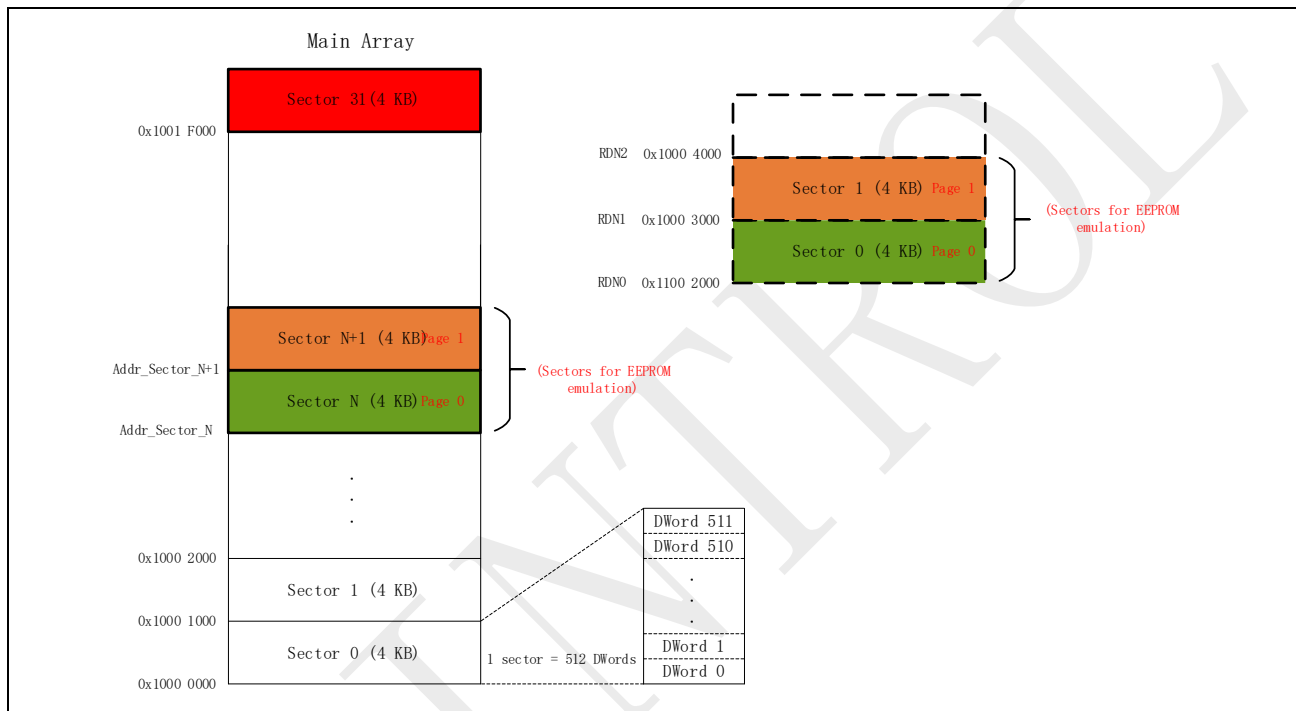
2.1.2 MEEPROM 实现

MEEPROM 使用 2 个 Page（Page0，Page1）来实现 E² PROM 模拟，而每个 Page 可以由 1 个或者多个 Flash Sector 组成。

当每个 Page 由 1 个 Flash Sector 组成时，算法会选择从 Addr_Sector_N（Addr_Sector_N 需要以 Sector 大小对齐）开始的连续 2 个 Sector 作为 Page0，Page1，如图 2-1 所示。

MEEPROM 可以用 Main Array 以及 RDN 区域中的 Sector 模拟 E² PROM。

图 2-2: MEEPROM 的 Page0 与 Page1

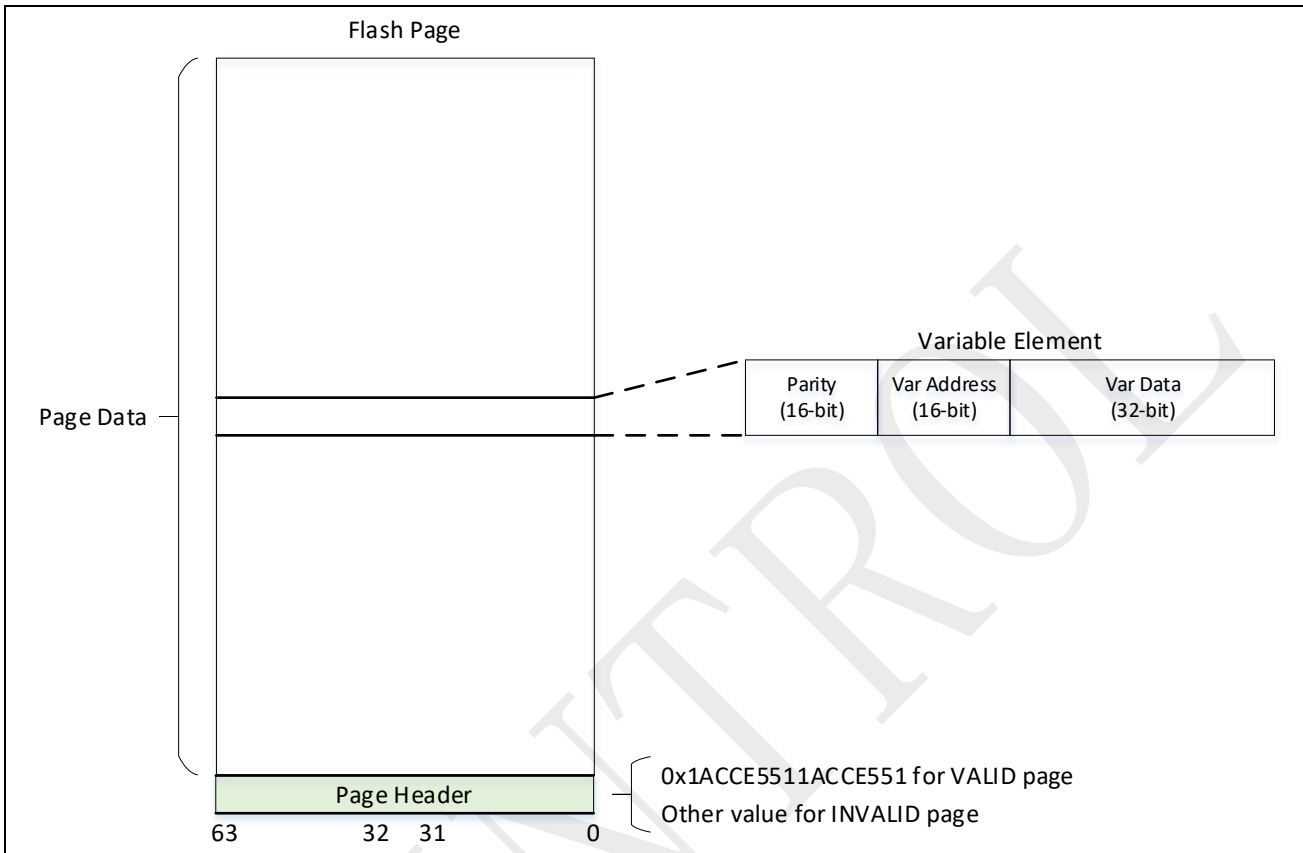


[1] DWord = Double Word, 1 DWord = 64-bit。

注意： MEEPROM 选择 Page 页时，不能用配置字（0x1001FFF8，0x1001FFFC）所在 Sector，否则会误锁调试接口，或误开看门狗，详见《技术参考手册》Flash 章节。

2.2 Page 格式

图 2-3: E²PROM 模拟算法中的变量格式



每个 Flash Page 的第一个 64 比特的双字 (Double-Word) 作为 Page Header，标识当前 Flash Page 的状态。每个 Flash Page 有下面两种可能的状态：

- **INVALID:** 该 Page 为空（初始状态，Page 擦除后即为空），或者正在接收其他 Page 搬移来的数据；
- **VALID:** 该 Page 包含有效数据，用于存储新数据；直到所有有效数据被搬移到另一个 Flash Page，该 Page 状态才会改变。

每个变量元素 (Variable Element) 都由一个 16 位校验 (Parity)，一个 16 位虚拟地址 (Var Address)，一个 32 位数据值 (Var Data) 来定义。

EEPROM 算法下，虚拟地址范围固定为 0 ~ 255。

MEEPROM 算法下，当每个 Page 由 1 个 Flash Sector 组成时，虚拟地址范围应固定为 0 ~ 255。

2.3 磨损均衡算法

磨损均衡算法允许监控和均匀分布不同 Flash Page 之间的写和擦除操作。当不使用磨损均衡算法时，Flash Page 不会以相同的频率使用。例如，具有长期数据的 Flash Page 不会像包含频繁更新数据的 Flash Page 那样经历那么多的写和擦除周期。磨损均衡算法可确保平等地使用每个 Flash Page 的所有可用的写周期。

根据设计，E²PROM 模拟算法在 Flash Page 之间均匀分配 Flash 写和擦除操作。无论什么用户变量被写入，Flash 写操作都按地址递增的顺序执行。磨损均衡算法具体实现如下：

- 当旧 Page 已满时，将旧 Page 中的有效变量元素复制到新 Page 中并将新 Page 设置为 VALID，然后擦除旧 Page。

2.4 原理示例说明

下面的示例展示了 3 个 E²PROM 变量的软件管理。变量的虚拟地址如表 2-2 所示。

表 2-2: 变量虚拟地址

变量	虚拟地址	数据位宽
Var1	0x01	32 位
Var2	0x04	32 位
Var3	0xFF	32 位

图 2-4 中, 展示了写数据过程。

图 2-4: 数据更新过程



翻 Page 过程:

1. 旧 Page 满;
2. 往新 Page 搬数据;
3. 往新 Page 写 VALID Header;
4. 擦除旧 Page。

在旧 Page 向新 Page 搬移数据（步骤 2）期间，如果突然断电，存在数据部分搬移状态。再次上电后，Init 会将 INVALID Page 擦除，并重新搬移。

在新 Page Header 设为 VALID 到旧 Page 擦除（步骤 3 到步骤 4）期间，如果突然断电，会存在新旧 Page Header 同时为 VALID 状态。再次上电后，对于 MEEPROM 算法（Page0, Page1），Init 通过 Page 内变量数目判断新旧 Page，并将旧 Page 擦除；对于 EEPROM 算法（Page0, Page1, Page2），按照 Page1 新于 Page0, Page2 新于 Page1, Page0 新于 Page2 判断两两新旧关系，并将旧 Page 擦除。

2.5 E²PROM 固件说明

应用程序可以调用的 E²PROM 固件函数如表 2-3 所示。

表 2-3: E²PROM 模拟算法 API 说明

函数名称	说明
Init	将模拟 E ² PROM 变量元素配置为其初始状态。 芯片复位后，在访问模拟 E ² PROM 之前，应该首先调用该函数。
Format	擦除所有用于模拟 E ² PROM 的 Flash Page，并初始化 E ² PROM。 Init 与 Format 均是初始化操作，调用 Format 后无需再调用 Init。
ReadWord	该函数用于获取虚拟地址对应的变量元素（最近一次存储）的数据值。 若找不到虚拟地址对应的变量元素，该函数返回 EEPROM_STATUS_NO_DATA。
WriteWord	该函数用于更新虚拟地址对应的变量元素的数据值。

2.6 E²PROM 模拟算法线程安全

E²PROM 模拟算法代码本身不保证线程间安全，需要保证冲突表 2-4 中的函数不在同一时刻运行。

表 2-4：冲突表函数

	Init	Format	WriteWord	ReadWord
Init	-	-	-	-
Format	-	-	-	-
WriteWord	-	-	-	-
ReadWord	-	-	-	-

表 2-5：冲突避免方法

	main 代码	中断
main 代码	无需特殊处理	main 代码调用冲突表内函数前需要采用 __disable_irq() 关中断响应； main 代码调用冲突表内函数后需要采用 __enable_irq() 开中断响应。
中断	main 代码调用冲突表内函数前需要采用 __disable_irq() 关中断响应； main 代码调用冲突表内函数后需要采用 __enable_irq() 开中断响应。	进入中断采用 __disable_irq() 关中断响应； 离开中断采用 __enable_irq() 开中断响应。

3 异常概述以及对应解决方案

表 3-1: E²PROM 模拟算法 API 返回值说明

返回值符号	返回值	描述
EEPROM_STATUS_OK	0x00000000U	操作成功
EEPROM_STATUS_WRITE_ERROR	0x00000001U	Flash 编程操作错误
EEPROM_STATUS_ERASE_ERROR	0x00000002U	Flash 擦除操作错误
EEPROM_STATUS_INVALID_ADDR	0x00000004U	虚拟地址参数值不在有效范围内： EEPROM 算法中，虚拟地址范围固定为 0 ~ 255； MEEPROM 算法中，当每个 Page 由 1 个 Flash Sector 组成时，虚拟地址范围固定为 0 ~ 255。
EEPROM_STATUS_WRITE_CHECK_FAIL	0x00000008U	写操作时，写入的变量元素回读校验错误
EEPROM_STATUS_NO_DATA	0x00000010U	读操作时，模拟算法找不到虚拟地址对应的变量元素
EEPROM_STATUS_INVALID_ENTRY	0x00000020U	写操作时，要写入变量的地址，不在当前有效的 Page 地址范围内 读操作时，要读取变量的地址，不在当前有效的 Page 地址范围内
EEPROM_STATUS_ELEMENT_NOT_EMPTY	0x00000040U	写操作时，要写入变量的地址内容不为空
EEPROM_STATUS_ADDR_MISMATCH	0x00000080U	读操作时，虚拟地址参数值与读取变量的虚拟地址不一致
EEPROM_STATUS_PARITY_ERROR	0x00000100U	读操作时，读取的变量 Parity 检验错误
EEPROM_STATUS_NO_PAGE_FOUND	0x00000400U	读/写操作时，未找到 VALID 的 Page
EEPROM_STATUS_PAGE_HEADER_ERROR	0x00000800U	设置 Page Header 为 VALID 时发生错误
EEPROM_STATUS_TRANSFER_ERROR	0x40000000U	Page 之间搬移数据错误，与其他错误返回值逻辑或，不独立存在
EEPROM_STATUS_INVALID_HEADER	0x80000000U	Page Header 无效组合： 未找到任何 VALID 的 Page； EEPROM 算法中，状态为 VALID 的 Page 达到 3 个； MEEPROM 算法中，状态为 VALID 的 Page 达到 2 个，但通过 Page 内变量数目无法判断新旧 Page。

3.1 EEPROM_STATUS_PARITY_ERROR

存在原因 1:

- 写过程被掉电打断，Flash 中存在脏数据，上电期间，对 VALID Page 中的数据进行检测，发现校验位和数据对不上，返回该状态。

处理方法:

- 控制写操作在掉电前完成，见[章节 5.1](#)。

存在原因 2:

- Flash 寿命到。

处理方法:

- 尝试调用 Format 彻底初始化 E² PROM 算法，失败则 return 1。

3.2 EEPROM_STATUS_INVALID_ADDR

存在原因 1:

- 写过程被掉电打断，Flash 中存在脏数据，上电期间，对 VALID Page 中的数据进行检测，发现虚拟地址位不在虚拟地址设定范围内，返回该状态。

处理方法:

- 控制写操作在掉电前完成，见[章节 5.1](#)。

存在原因 2:

- 例如初始化时设定虚拟地址范围 0-255，但执行 WriteWord 时，尝试往虚拟地址 300 处写数据（软件 Error）。

处理方法:

- 修正 WriteWord 传入虚拟地址到设定虚拟地址范围内。

存在原因 3:

- 例如初始化时设定虚拟地址范围 0-255，但执行 ReadWord 时，尝试读虚拟地址 300 处数据（软件 Error）。

处理方法:

- 修正 ReadWord 传入虚拟地址到设定虚拟地址范围内。

3.3 EEPROM_STATUS_NO_DATA

存在原因 1:

- 例如之前从未往虚拟地址 125 处写过任何数据，但执行 ReadWord 时，尝试读取虚拟地址 125 处数据（软件 Error）。

处理方法:

- 修正 ReadWord 传入虚拟地址到有效虚拟地址范围内。

3.4 EEPROM_STATUS_INVALID_CB

存在原因 1:

- MeeepromCtrlInfo.BASE_ADDR 超出 Main Flash 可用范围;
- MeeepromCtrlInfo.u32MaxVarNum 超出 Page Date 空间一半（软件 Error）。

处理方法:

- MeeepromCtrlInfo.BASE_ADDR 调整到 Main Flash 可用范围;
- MeeepromCtrlInfo.u32MaxVarNum 调整到 Page Date 空间一半以下。

3.5 EEPROM_STATUS_INVALID_HEADER

存在原因 1:

- 发生在首次下载 E²PROM 程序调用 Init 时，发现所有 Page Header INVALID（软件 Error）。

处理方法:

- 首次下载 E²PROM 程序调用 Init 时，发现 EEPROM_STATUS_INVALID_HEADER，则调用 Format，实现全擦初始化，失败则 return 1。

存在原因 2:

- MeeepromCtrlInfo.u32MaxVarNum 过大，Init 无法通过 Page 内变量数量判断新旧 Page（软件 Error）。

处理方法:

- MeeepromCtrlInfo.u32MaxVarNum 调整到 Page Date 空间一半以下。

存在原因 3:

- Flash 寿命到。

处理方法:

- 尝试调用 Format 彻底初始化 E²PROM，失败则 return 1。

3.6 EEPROM_STATUS_NO_PAGE_FOUND

存在原因 1:

- 调用了非 E²PROM 函数擦除属于 E²PROM 的 Flash 区域，导致 E²PROM 算法信息丢失，进而导致 ReadWord, WriteWord 执行期间发现所有 Page Header 均为 INVALID (软件 Error)。

处理方法:

- 删除程序中非 E²PROM 逻辑擦 E²PROM Flash 区域的操作。

存在原因 2:

- Flash 寿命到。

处理方法:

- 尝试调用 Format 彻底初始化 E²PROM，失败则 return 1。

3.7 EEPROM_STATUS_ADDR_MISMATCH

存在原因 1:

- 变量申请大空间内存造成栈溢出，或递归层次太深，MeepromCtrlInfo.pEntryTable (虚拟地址与物理地址对照关系) 与物理地址中实际存储的虚拟地址不对应 (软件 Error)。

处理方法:

- 避免变量申请大空间内存;
- 减少或避免使用递归代码;
- 增大栈空间。

存在原因 2:

- Flash 寿命到。

处理方法:

- 尝试调用 Format 彻底初始化 E²PROM，失败则 return 1。

3.8 EEPROM_STATUS_TRANSFER_ERROR

存在原因 1:

- 其它 Error 产生，永远和某个具体 Error 一起产生。

处理方法:

- 采用“&”操作获取具体 Error 并处理，无需处理 EEPROM_STATUS_TRANSFER_ERROR 本身。

3.9 EEPROM_STATUS_WRITE_ERROR

存在原因 1:

- Flash 存在写保护（软件 Error）。

处理方法:

- 解除 E²PROM Flash 区域写保护。

存在原因 2:

- Flash 寿命到。

处理方法:

- 尝试调用 Format 彻底初始化 E²PROM，失败则 return 1。

3.10 EEPROM_STATUS_ERASE_ERROR

存在原因 1:

- Flash 存在写保护（软件 Error）。

处理方法:

- 解除 E²PROM Flash 区域写保护。

存在原因 2:

- Flash 寿命到。

处理方法:

- 尝试调用 Format 彻底初始化 E²PROM，失败则 return 1。

3.11 EEPROM_STATUS_PAGE_HEADER_ERROR

存在原因 1:

- 设置 Page Header 为 VALID 失败，唯一的可能性是 FLASHC_ProgramDWord 执行失败。

处理方法:

- 尝试调用 Format 彻底初始化 E²PROM，失败则 return 1。

3.12 EEPROM_STATUS_ELEMENT_NOT_EMPTY

存在原因 1:

- 正常使用时不会出现该错误状态，如果出现，则可能是违反了表 2-4 的规定（软件 Error）。

处理方法:

- 见表 2-5。

3.13 EEPROM_STATUS_WRITE_CHECK_FAIL

存在原因 1:

- 正常使用时不会出现该错误状态, 如果出现, 则可能是违反了表 2-4 的规定 (软件 Error)。

处理方法:

- 见表 2-5。

3.14 EEPROM_STATUS_INVALID_ENTRY

存在原因 1:

- 正常使用时不会出现该错误状态, 如果出现, 则可能是违反了表 2-4 的规定 (软件 Error)。

处理方法:

- 见表 2-5。

4 固件对应解决方案

确保[章节 0](#) 软件 Error 正确处理后，可为异常分支添加 Format 操作，确保存在 Flash 硬件故障时，脏数据能够被完全清除。

图 4-1: 函数 ReadWord

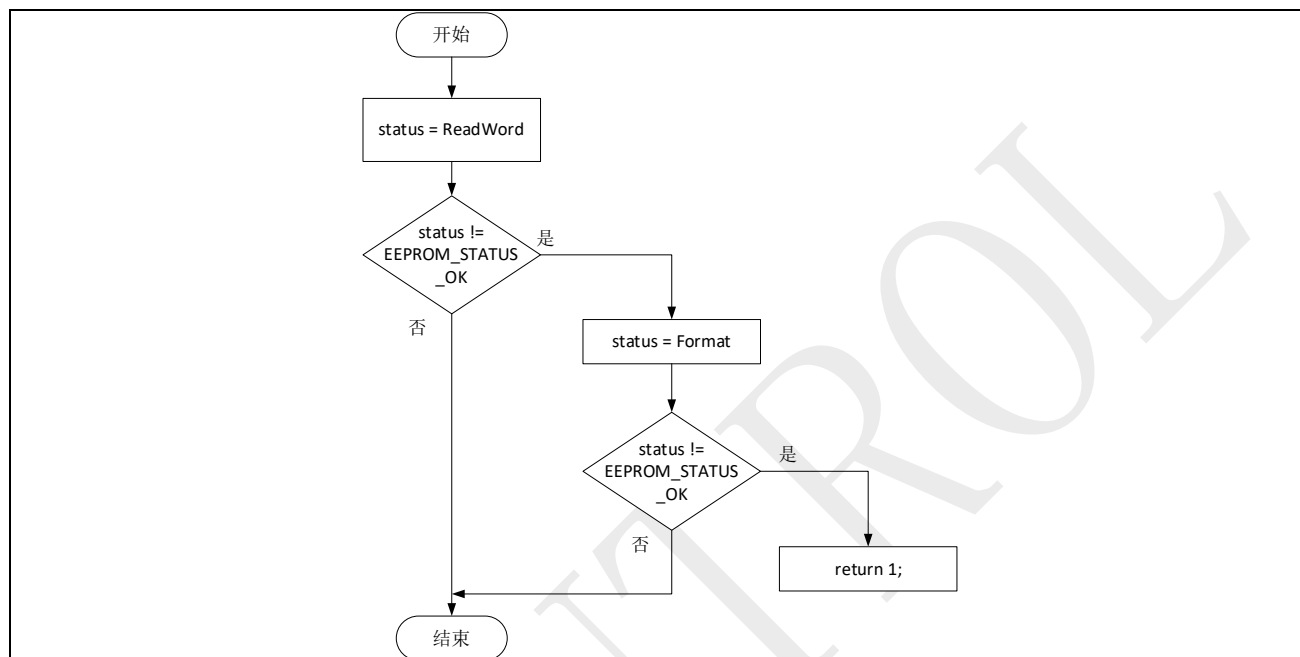


图 4-2: 函数 WriteWord

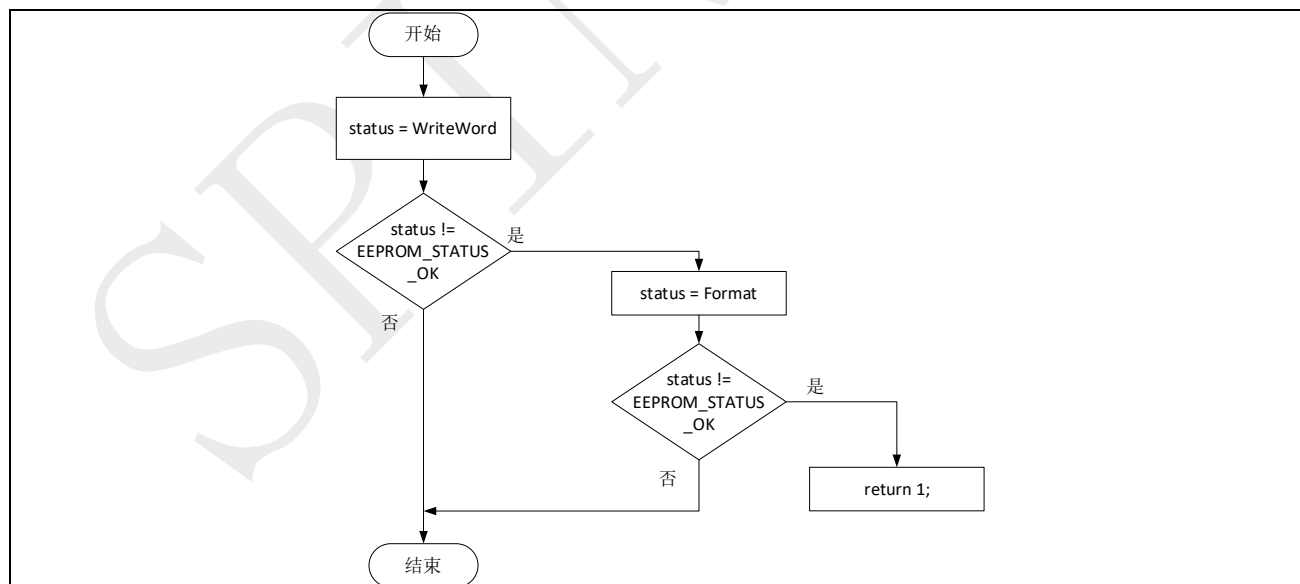


图 4-3：函数 Init

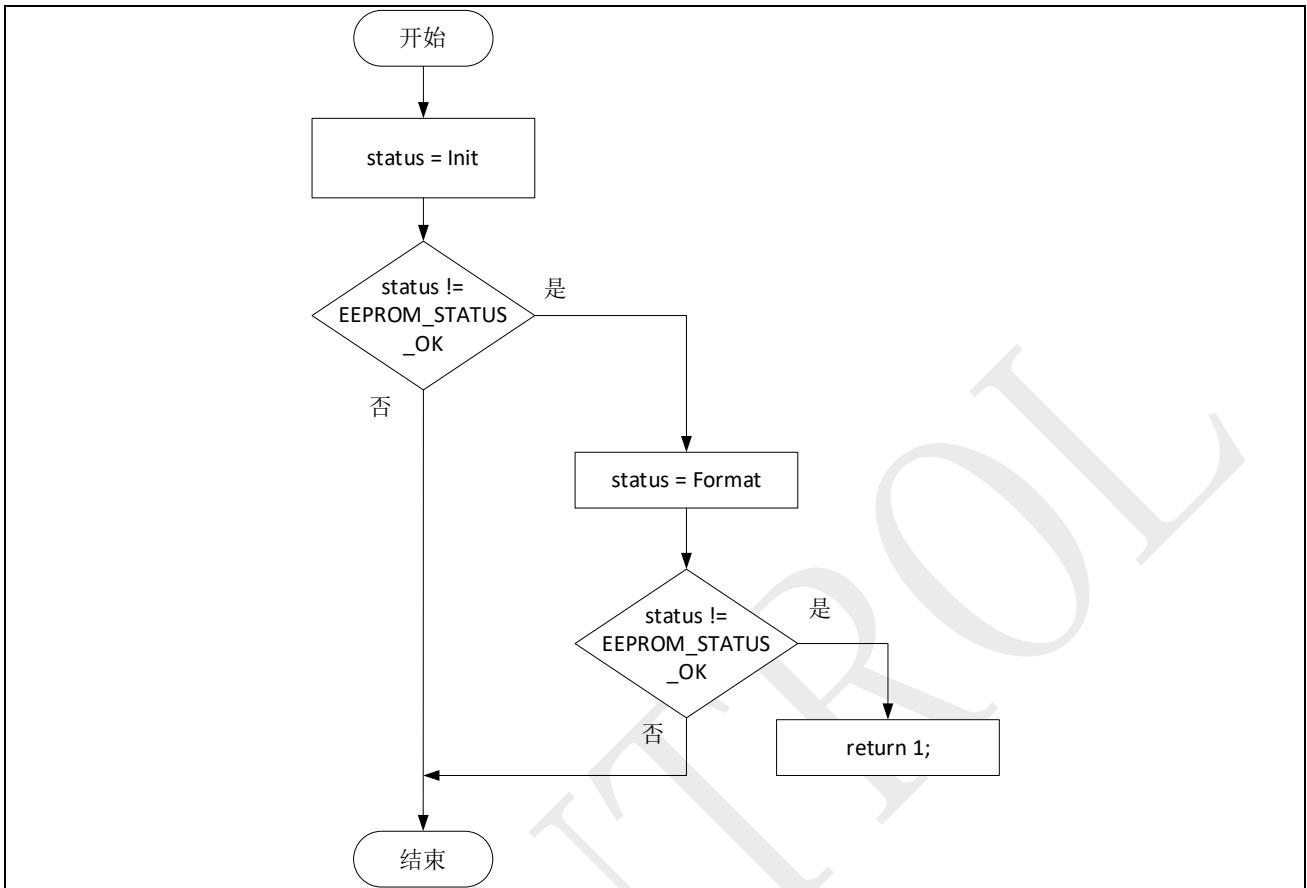
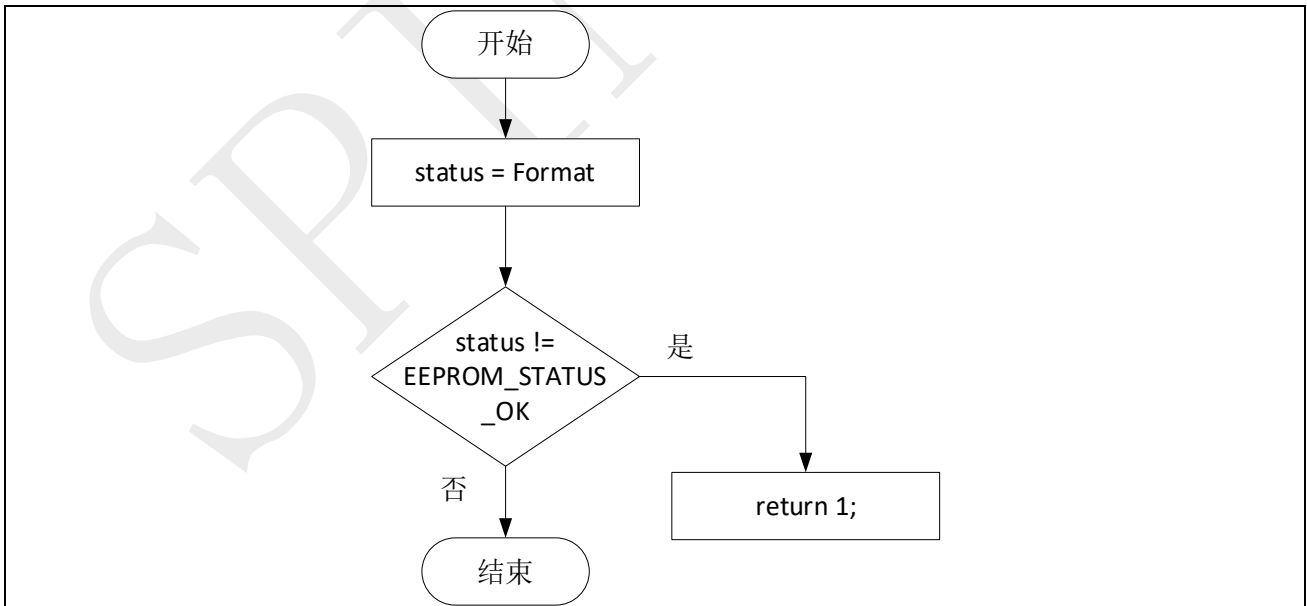


图 4-4：函数 Format



5 工程应用讨论

本章节将针对不同应用场景所应注意的事项展开讨论。

5.1 掉电保存数据

在实际工程应用中，通常存在掉电保存数据的需求，掉电的可能原因有：主动掉电、随机掉电。对于 SPD1179 平台的产品而言，通常需要保证 E² PROM 模拟算法的写操作在电压掉电至 5.5V 前完成，否则数据将有可能写入不成功。

5.1.1 主动掉电

若工程中是主动调用 SLEEP 或 STOP 指令，此时属于主动掉电，为了确保数据能成功写入 E²PROM，则需按照如下步骤操作：

- 先调用 WriteWord 将数据保存；
- 再调用 SLEEP 以及 STOP 命令，使系统进入低功耗。

5.1.2 随机掉电

5.1.2.1 SPD1179/SPD1176

若工程中存在电源故障导致系统掉电的风险，此时为了确保数据能够成功写入模拟的 E² PROM，则需按照如下步骤操作：

- 通过 VBAT 欠压中断 EPWRTZO_IRQHandler 检测电源故障；
- 从表 1-2 中可以看出，写一个 Word（32-bit）的时间从 50us 到 18ms（伴有 Flash Page 数据搬移工况）不等。这意味着在随机掉电的应用场景中，如果要使用 E² PROM 模拟算法，需要考虑电容的电量是否能够满足数据写入的时间要求；若实际工程中，由于各种因素考量，使得电容电量无法满足最坏工况下一个 Word（18ms）写入时长的需求，则可考虑如下解决办法：
 - 增加 VBAT 欠压中断触发阈值，从而延长 VBAT 欠压中断触发到 VBAT 下降到 5.5V 的时间；
 - 参考 SDK 中 E²PROM_Data_Storage_Upon_VBAT_UV_Event 示例工程提供的解决思路。其将数据搬移工况（18ms）从掉电场合，变更到上电场合，从而将掉电时 ms 级别写入时间缩减到 us 级。

5.1.2.2 SPC1169

不同于 SPD1179，SPD1176，VBAT 从 28V 下降到 5.5V 期间，VDD33 能一直维持在 3.3V（维持 MCU 正常工作电压），SPC1169 直接采用 VDD33 供电，实际掉电过程（VDD33 下降到 2.97V 以下）极易引发模拟 E²PROM 工作异常（因为 VDD33 对应正常工作电压与最小工作电压的差值很小，如表 5-1 所示）。

因此，SPC1169 如使用模拟 E²PROM，必须将整个数据写入过程严格控制在 VDD33 下降到 2.97V 之前。

表 5-1：V_{DVDD33} 推荐工作条件

符号	参数	条件	最小值	正常值	最大值	单位
V _{DVDD33}	3.3V 数字电压	-	2.97	3.3	3.63	V

5.2 数据粒度管理

E² PROM 算法可用于需要以字节、半字（Half-Word）或字（Word）粒度更新数据的非易失性存储的嵌入式应用。数据大小通常取决于应用要求，例如传感器或通信接口数据大小。

E² PROM 算法设计用于支持 32 位字粒度。对于字节、16 位半字的非易失性存储需求，应用程序可以将其转换成 32 位字类型，然后再存储到模拟 E² PROM 中。但是，这种方式对 Flash 的存储空间利用率较低。为了优化 Flash 的使用，用户应用程序可以将所有较小尺寸的数据元素收集到 32 位数据元素中，然后再将内容存储在模拟 E² PROM 中。这将通过同时写入 16 位虚拟地址、16 位 Parity 和 32 位数据值来确保 64 位 Flash 的最佳使用。

5.3 XIP 关闭期间中断处理

调用 Init，Format，WriteWord 擦或写 Flash 期间，均会关闭 Flash XIP，期间任何中断执行均会失败。

解决方法任选其一：

- 方法一：在调用这类前关闭中断响应（__disable_irq()），在调用这类函数后重新开启中断响应（__enable_irq()）；
- 方法二：将中断向量表，以及所有中断函数放到 RAM 中，参见例程《18_5_Flash_With_ISR_In_RAM》。

5.4 看门狗不断复位或芯片调试接口被锁定

使用 Flash 时，不能用配置字（0x1001FFF8， 0x1001FFFC）所在 Sector，否则会误锁调试接口，或误开看门狗，详见《技术参考手册》中 Flash 章节。

5.5 中断时间开销

对时间敏感的应用场景，需要减少中断数量，降低时间开销。

对同一中断，逻辑代码完全相同，需要减少先后触发次数。

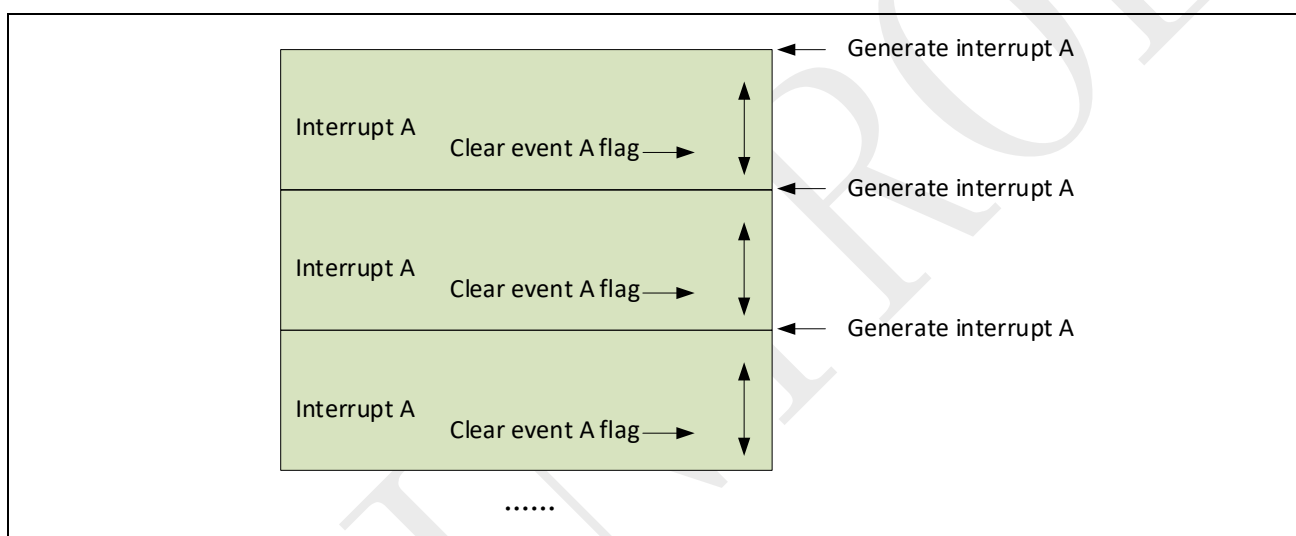
对不同中断，如果逻辑代码类似，比如均要往模拟 E² PROM 写数据，也要避免先后触发。

5.5.1 同一中断不断触发以及对应解决方案

存在原因 1:

- 当中断触发条件持续存在时，中断会一直触发的。

图 5-1: 中断 A 先后触发



处理方法:

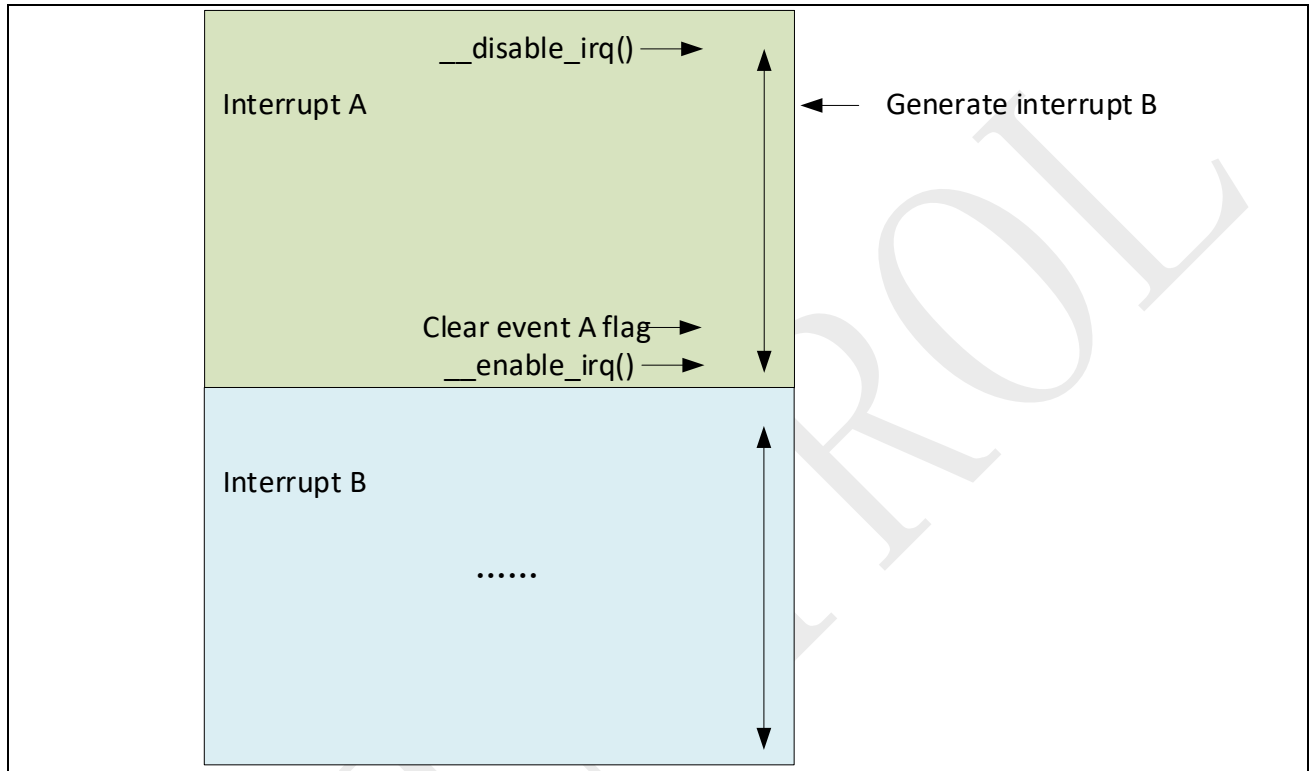
- 如果不需要对中断反复响应，可通过 Timer 在一定时间内关闭中断。

5.5.2 不同中断先后触发以及对应解决方案

存在原因 1:

- `__disable_irq()` 只能用来保证中断顺序执行，也就是线程安全。`__enable_irq()` 后，MCU 会立即处理之前触发的中断。

图 5-2: 中断 A, B 先后触发



处理方法:

- 可通过在中断 A 中清中断 B Flag 来清除 B 事件。

图 5-3: 在中断 A 中清中断 B Flag

